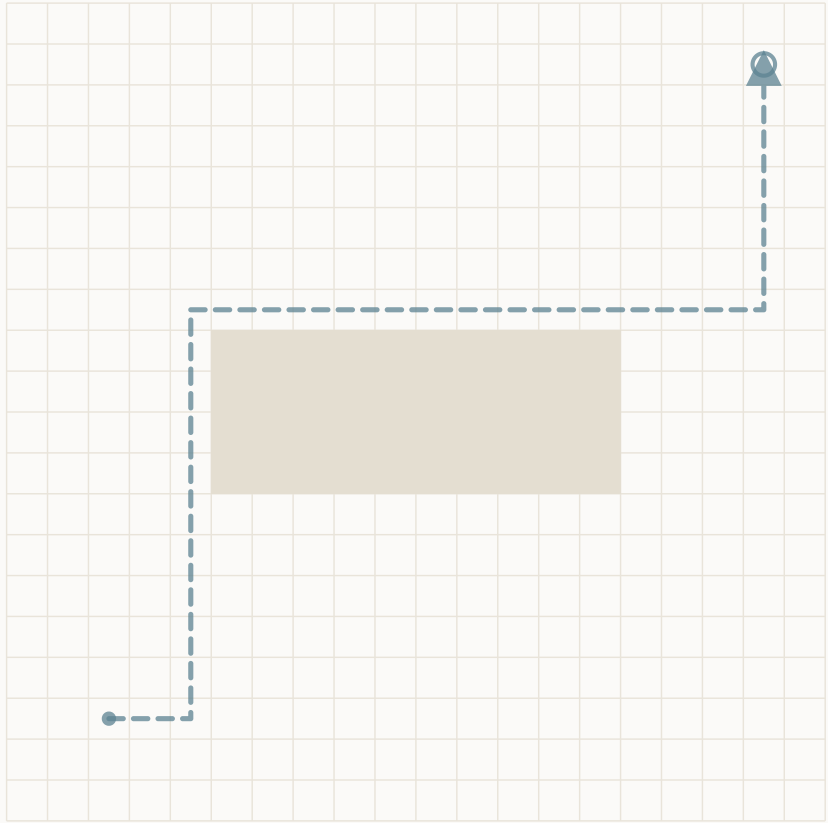
Navegación de robots móviles

Capítulo 5 — de lo reactivo al mapa global

Juan David · Ingeniería Electrónica · UNAL Manizales
Robótica · Semestre VIII · 2026-2



Agenda

1. **Contexto y banco de pruebas**
2. **Parte A** — Braitenberg
3. **Parte B** — Bug2
4. **Parte C** — A*
5. **Parte D** — distance transform
6. **Síntesis y conclusiones**

> La pregunta que atraviesa las cuatro partes: **qué información necesita el robot** —y qué paga por tenerla— para llegar al objetivo.

1. Contexto

La guía cubre cinco partes; esta entrega resuelve cuatro

Alcance del laboratorio

Parte	Método	Estado
A	Braitenberg	✅ Implementada
B	Bug2 simplificado	✅ Implementada
C	A*	✅ Implementada
D	Distance transform	✅ Implementada
E	ROS2 / Nav2	⏳ Entrega posterior

Por qué la parte E queda fuera

Integrar Nav2 exige el stack de ROS2, `nav2_bringup` y un robot simulado. La instrucción de esta sesión cubre A–D.

Consecuencia práctica

Las cuatro simulaciones son locales: `numpy` + `matplotlib`, sin ROS2 y reproducibles con un solo comando.

> Doce preguntas de análisis respondidas con los resultados de las corridas y el texto de Corke.

El banco de pruebas común

Mapa

- Grid de **20 × 20** celdas
- Obstáculo `grid[8:12, 5:15] = 1` → 4 × 10 celdas
- **360** celdas libres, **40** ocupadas

Coordenadas

- Celda `(x, y)` ↔ `grid[y, x]`
- Figuras con `origin="lower"` : y crece hacia arriba

Grafo de búsqueda

- **4 vecinos** (arriba, abajo, izquierda, derecha)
- Costo unitario por arista, **sin diagonales**
- Inicio $(2, 2)$, objetivo $(18, 18)$

La cota que usaremos como referencia

$$d_{\text{Manhattan}} = |18 - 2| + |18 - 2| = 32$$

> Ningún camino de 4 vecinos puede medir menos de 32 pasos. Ese número es el árbitro de optimalidad en este mapa.

2. Parte A — Braitenberg

Dos sensores, una resta, ninguna memoria

Una ley de control puramente reactiva

El campo escalar tiene su máximo en $(60, 90)$:

$$s(x, y) = \frac{1000}{(x - 60)^2 + (y - 90)^2 + 50}$$

Dos sensores separados $d = 2$ respecto al eje del robot:

$$v = -K_v \left(s_L + s_R\right)$$
$$\omega = K_s \left(s_R - s_L\right)$$

Parámetros

Parámetro	Valor
K_s	17
K_v	14
dt	0.1 s
Estado inicial	$(10, 10, 0^\circ)$
Iteraciones	$T = 222$

Cinemática del robot diferencial simplificado:

Trayectoria: un arco, no una recta

- Parte de $(10, 10)$ con $\theta = 0$
- Se mueve **hacia la izquierda**: en x se aleja del objetivo
- Describe un arco amplio y termina en $(51.36,\ 85.82)$
- Distancia final con $T = 222$: **9.60 celdas**
- Nunca apunta directamente al objetivo

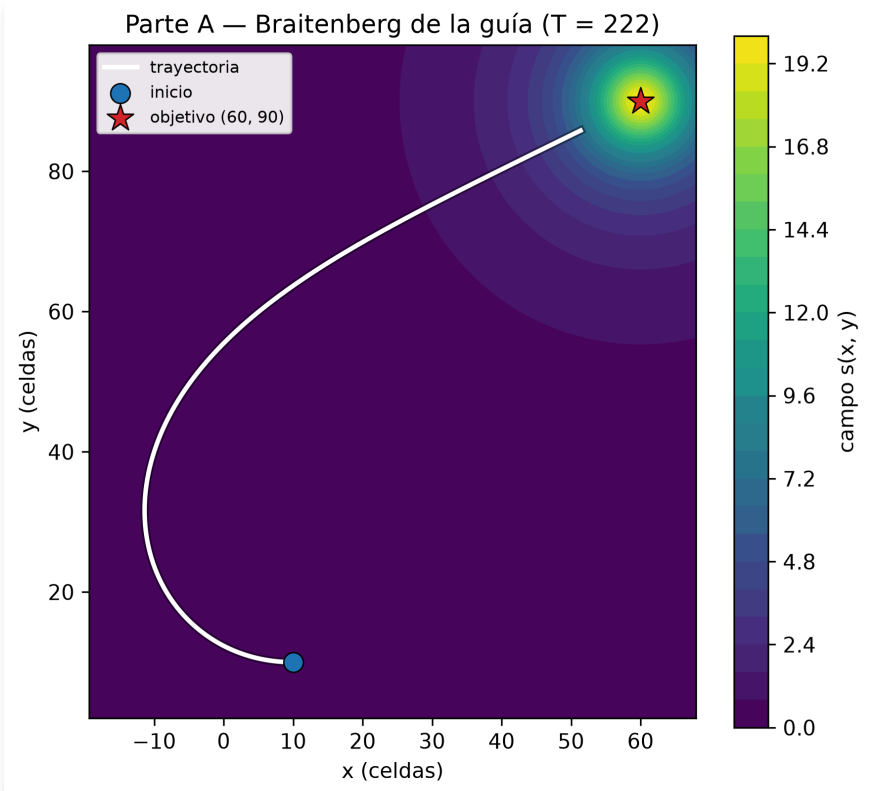
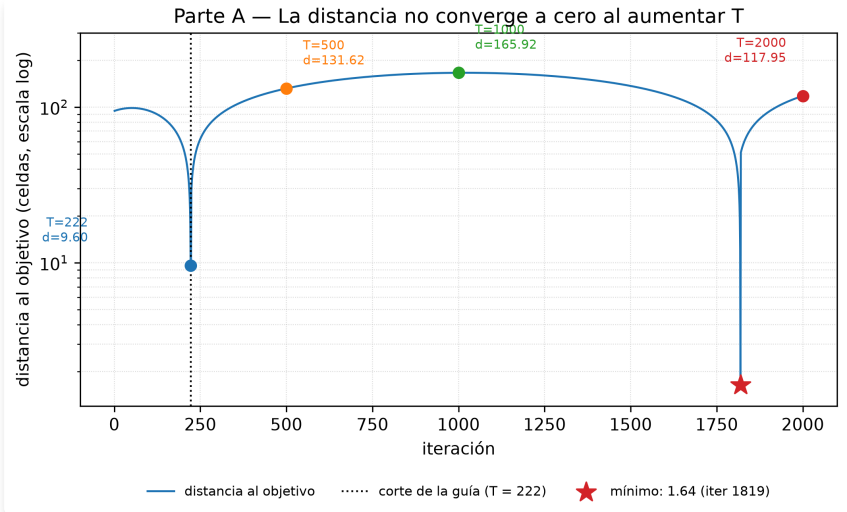


Figura A1 — campo del sensor y trayectoria para T = 222

Más iteraciones no arreglan la ley

T	$d_{\min}$	Iter. del mín.	d_{final}
222	9.596	222	9.596
500	9.570	223	**131.62**
1000	9.570	223	**165.92**
2000	1.642	1819	117.95



La curva completa: el robot **pasa cerca** del objetivo y sigue de largo. El mínimo global (1.64 en la iteración 1819) es una casualidad de la órbita, no una llegada.

Figura A2 — distancia al objetivo sobre la trayectoria completa

Diagnóstico: por qué no converge

1. El signo

$v = -K_v(s_L + s_R)$ crece en magnitud como $1/r^2$ al acercarse. En el paso siguiente a la mejor posición, $|v| \approx 192$ → salto de **19.2 celdas** en un solo dt.

2. No hay frenado ni parada

Cerca del objetivo $s_L \approx s_R$, así que $\omega \to 0$: el robot deja de girar, pero **no de avanzar**. Sobrepasa y se aleja.

3. No hay mapa ni evasión

El campo es global y conocido. Un obstáculo físico lo bloquearía: no hay sensores de proximidad ni comportamiento de evasión.

La ley del libro sí converge: $v = 2 - s_R - s_L$. En el objetivo $s_R = s_L = 1$, entonces $v \to 0$ y el robot frena de forma asintótica. La guía cambió el signo y perdió esa propiedad.

3. Parte B — Bug2

Estado y evasión de obstáculos, todavía sin mapa

El algoritmo, en cuatro pasos

1. Calcular la dirección al objetivo
2. Avanzar por el **eje de mayor diferencia** (sin diagonales)
3. Si la celda siguiente está ocupada, probar los vecinos en orden **derecha** → **arriba** → **izquierda** → **abajo** y moverse al primero libre
4. Registrar la posición; terminar a menos de 1 celda del objetivo

> **Versión simplificada:** no calcula la m-line ni usa la condición de salida del Bug2 completo. Esa omisión explica todo lo que falla después.

```
for it in range(1, max_iter + 1):
    direction = goal - pos

    # eje dominante: nunca diagonal
    if abs(direction[0]) > abs(direction[1]):
        step = (sign(direction[0]), 0)
    else:
        step = (0, sign(direction[1]))

    nxt = pos + step
    if is_free(grid, nxt):
        pos = nxt
    else:
        # fallback fijo: derecha, arriba,
        # izquierda, abajo
        for move in moves:
            if is_free(grid, pos + move):
                pos = pos + move
                break
```

Rodea el obstáculo y llega

Métrica	Valor
Longitud del camino	**32 pasos**
Longitud euclidiana	32.0 celdas
¿Llegó?	Sí, 32 iteraciones
Cota inferior de Manhattan	32

El camino baja hasta $y = 7$ (justo debajo del obstáculo) y sube por $x = 15$ (justo a la derecha). La m-line punteada se dibuja **solo como referencia**: el código no la usa.

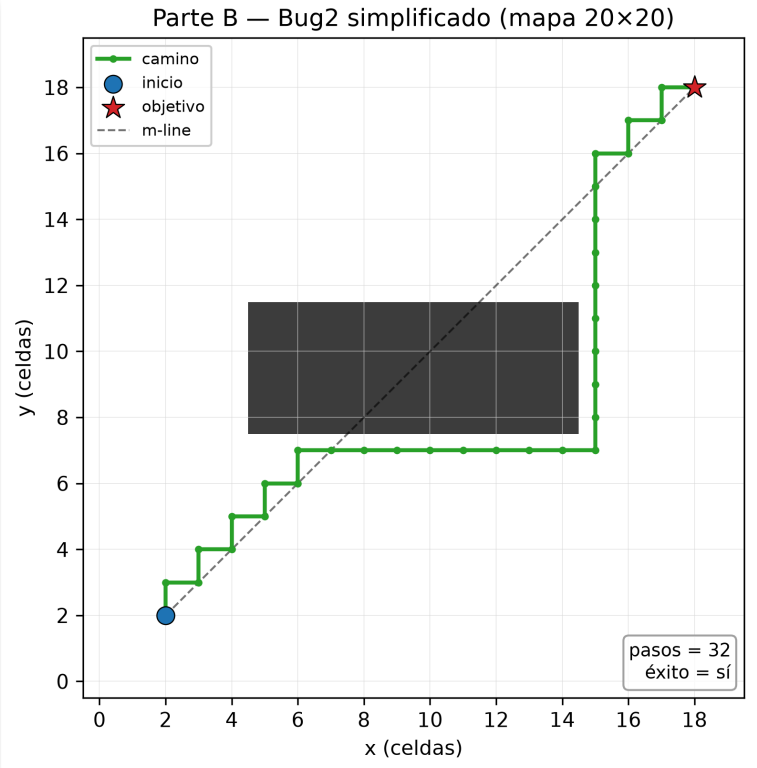


Figura B1 — camino del Bug2 simplificado

¿Es óptimo? Sí aquí, no en general

En este mapa, sí

Cada paso cambia una coordenada en una unidad, así que cualquier camino necesita al menos $|18-2|+|18-2| = 32$ pasos. El camino mide exactamente 32: alcanza la cota y es óptimo.

Pero no es una propiedad del algoritmo

Bug2 es local: sigue el borde sin visión global. En la casa del libro recorre **1307** puntos frente a **338** del distance transform — casi cuatro veces más.

La prueba que lo delata — mismo mapa, inicio y objetivo invertidos $(18, 18) \to (2, 2)$:

- Se agotan las 500 iteraciones: **no llega**
- Visita solo **13 celdas distintas**
- Queda ciclando entre $(12, 12)$ y $(13, 12)$

En (12,12) el empate |Δx| = |Δy| hace que intente bajar —obstáculo—; el fallback lo manda a la derecha y desde allí el paso dominante lo devuelve. Sin condición de salida, el ciclo es inevitable.

A contra B: la diferencia es el estado

Aspecto	A — Braitenberg	B — Bug2 simplificado
¿Mapa global?	No	No
¿Memoria / estado?	No	Sí: camino y detección de obstáculos
¿Detecta obstáculos?	No	Sí: consulta celdas vecinas
¿Llega al objetivo?	No (mínimo 9.57)	Sí, en 32 pasos
Trayectoria	Continua, orbital	Discreta, en escalera
Optimalidad	No	Aquí sí; en general no
Entornos complejos	Se bloquea con un obstáculo	Puede ciclar (versión simplificada)

> El Braitenberg conecta sensores y motores **sin memoria**. El Bug2 añade una máquina de estados (avanzar / seguir borde) y recuerda el camino. Esa memoria resuelve obstáculos, pero sigue siendo navegación reactiva.

4. Parte C — A*

El mapa como grafo y una heurística que dirige la búsqueda

A*: costo recorrido + costo estimado

Criterio de expansión

$$f(n) = g(n) + h(n)$$

- $g(n)$: costo acumulado desde el inicio
- $h(n)$: estimación del costo restante
- Cola de prioridad por f
- h euclidiana: $h(n) = \|n - \text{goal}\|_2$

Detalle de implementación

No usa conjunto cerrado: si encuentra un costo mejor, **vuelve a encolar** el nodo. Eso lo hace robusto ante heurísticas inconsistentes.

```
def astar(grid, start, goal):
    push(open_list, (0.0, start))
    cost = {start: 0}

    while open_list:
        _, current = heappop(open_list)
        if current == goal:
            break
        for dx, dy in FOUR_NEIGHBORS:
            neighbor = current + (dx, dy)
            if not free(grid, neighbor):
                continue
            new_cost = cost[current] + 1
            if neighbor not in cost \
               or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                priority = new_cost + h(neighbor, goal)
                push(open_list, (priority, neighbor))
                came_from[neighbor] = current
```

Treinta y dos pasos, 305 nodos expandidos

Métrica	Valor
Longitud del camino	**32 pasos**
$g(\text{objetivo})$	32 aristas
Nodos extraídos	**305**
Nodos encolados	347
Tiempo (indicativo)	≈ 1.2 ms

A* sube por $x = 4$ hasta $y = 12$, cruza **por encima** del obstáculo y avanza en escalera hasta $(18, 18)$.

Es una ruta distinta de la de Bug2 —que pasa por debajo— pero mide lo mismo: **32 pasos**.

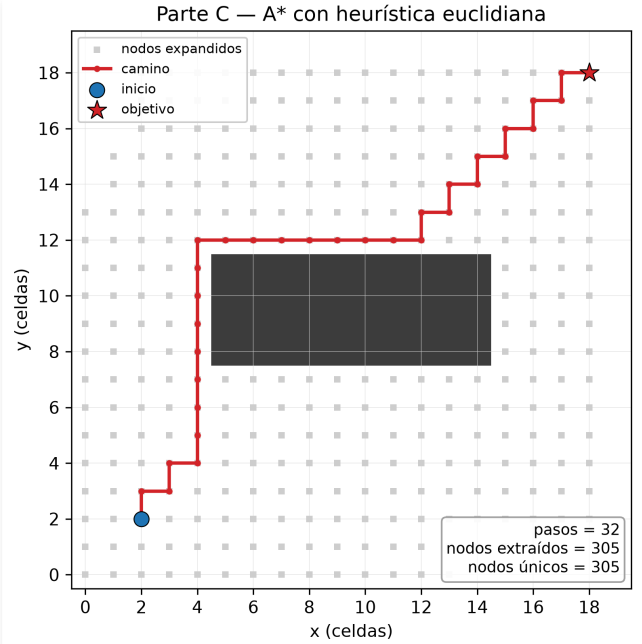


Figura C1 — camino y nodos expandidos (gris)

BFS, UCS y A*: tres formas de ordenar la cola

Algoritmo	Criterio de expansión	Qué garantiza	Comentario
BFS	Cola FIFO por número de aristas	Menos aristas, si todas cuestan lo mismo	Ignora costos; explora mucho
UCS	Cola de prioridad por $g(n)$	Costo mínimo con costos no negativos	Onda en todas las direcciones
A*	Cola de prioridad por $f = g + h$	Costo mínimo si h es admisible	Dirige la búsqueda; expande menos

Relaciones

BFS es UCS cuando todas las aristas cuestan 1.
UCS es A* cuando $h(n) = 0$.

Evidencia del libro

Mismo camino óptimo de costo 1659: UCS explora **19** vértices, A* explora **12**.

La heurística: admisible, consistente, y aun así floja

Qué exige la optimalidad

- **Admisible:** $h(n) \leq h^*(n)$, nunca sobreestima
- **Consistente:** $h(n) \leq c(n, n') + h(n')$

La euclidiana cumple ambas en un grid de 4 vecinos.

Por qué no basta con ser correcta

En 4-conectividad la euclidiana **subestima mucho**: un desplazamiento diagonal cuesta 2 pasos, no $\sqrt{2}$. Muchos nodos empatan en f.

Resultado: A* expande **305 de 360** celdas libres — el **85 %**.

Qué pasaría con otra heurística

Heurística	Efecto
Manhattan	Admisible y más ajustada: menos expansiones
$h = 0$	A* se degrada a UCS: explora más

5. Parte D — Mapas de ocupación

Calcular la distancia al objetivo para todas las celdas

Distance transform: la solución en todo el mapa

Inicialización

$$D(\text{meta}) = 0, \qquad D(c) = \infty \ \text{ para } c \neq \text{meta}$$

Actualización por barridos

$$D(y,x) \leftarrow \min\Big(D(y,x),\ 1 + \min_{n \in N(y,x)} D(n)\Big)$$

Punto fijo: ecuación de Bellman

$$D(\text{meta}) = 0, \qquad D(c) = 1 + \min_{n} D(n)$$

Cómo se lee

Cada celda guarda **cuántos pasos le faltan** para llegar a la meta. Al llegar al punto fijo, basta con bajar por el gradiente: desde cada celda, ir al vecino con menor D.

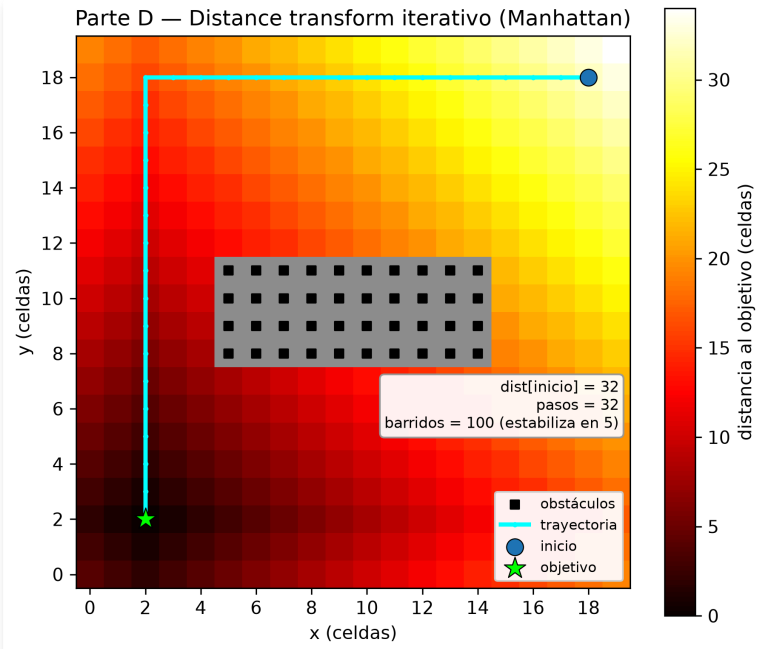
Detalles de la guía

- 4 vecinos, costo 1 → métrica **Manhattan** (L_1)
- Obstáculos quedan en ∞
- **100 barridos fijos**, sin verificar convergencia

> El mapa de distancias se calcula **una vez** y sirve para cualquier inicio: es la diferencia con A*, que busca por consulta.

Un mapa de calor de distancias

Métrica	Valor
Longitud del camino	**32 pasos**
$D(\text{inicio})$	**32 celdas**
Barridos ejecutados	100
Barrido de estabilización	**5**
Celdas libres / obstáculos	360 / 40
Tiempo (indicativo)	≈ 31 ms



La trayectoria sale de $(18, 18)$, recorre el borde superior hasta $(2, 18)$ y baja por el borde izquierdo hasta $(2, 2)$.

Coincide con $D(\text{inicio}) = 32$: la distancia almacenada **es** la longitud del camino.

Por qué el resultado es óptimo

Sea L la longitud de cualquier camino desde c hasta la meta. Por inducción en los pasos:

$$D(c) = 1 + \min_n D(n) \;\leq\; 1 + D(n') \;\leq\; 1 + (L-1) = L$$

Es decir, $D(c)$ es una **cota inferior** de cualquier camino. Además, el camino que baja por el vecino de menor D reduce D exactamente en 1 en cada paso, así que alcanza la cota: **es óptimo**.

Por qué la guía dice "NO ÓPTIMO"

1. Con 4 vecinos optimiza **Manhattan**, no la distancia euclidiana.
2. Ejecuta **100 barridos fijos** sin verificar convergencia.

La lectura correcta

La optimalidad es del **punto fijo** y de la **métrica modelada**. En este mapa se alcanza en 5 barridos y el camino de 32 pasos es óptimo en L_1 — no la ruta euclidiana más corta para un robot con diagonales.

Costo: bueno en 20 × 20, insuficiente en la casa

Complejidad

- Cada barrido: $O(N^2)$ (una visita por celda, 4 vecinos)
- Barridos: al menos $O(N)$ para propagar la información
- Costo típico: $O(N^3)$; peor caso (laberinto): $O(N^4)$
- Memoria: $O(N^2)$

En este mapa

100 barridos × 400 celdas × 4 vecinos ≈ 1.6×10^5 actualizaciones ≈ 31 ms. **95 de los 100 barridos no cambian nada**: el mapa se estabiliza en 5.

La casa del libro (397 × 596)

Dato	Valor
Barridos necesarios	**571**
Celdas inalcanzables	9925
Bug2 (completo)	1307 pasos
Distance transform	**338 pasos**

Con el límite fijo de 100 barridos, el código de la guía **ni siquiera convergería**. El número de barridos debe adaptarse al mapa o detectar la convergencia y detenerse.

Síntesis: el precio de saber más

Parte	Método	¿Mapa?	¿Memoria?	¿Óptimo?	Resultado 20×20
A	Braitenberg	No	No	No	d-mín = 9.57; no llega
B	Bug2 simplificado	No	Sí (camino)	Aquí 32; en general no	32 pasos
C	A*	Sí	Sí (cola y costos)	Sí, con h admisible	32 pasos; 305 expansiones
D	Distance transform	Sí	Sí (mapa de D)	Sí (Bellman, L1)	32 pasos; D(inicio) = 32

A → B
Se añade **estado**: aparece la evasión de obstáculos.

B → C/D
Se añade **mapa global**: aparece la optimalidad.

C ↔ D
A* busca solo lo necesario; el DT resuelve **para todas** las celdas.

Conclusiones (1/2)

1. La ley de Braitenberg de la guía no converge. Velocidad negativa sin saturación + ausencia de condición de parada = sobrepaso. Con T = 500 termina a 131.62 celdas; con T = 2000, a 117.95. Aumentar iteraciones solo alarga la órbita. La ley del libro ($v = 2 - sR - sL$) sí frena porque $v \to 0$ en el objetivo.

2. El Bug2 simplificado es óptimo aquí, pero no en general. 32 pasos alcanza la cota de Manhattan en este mapa. Sin embargo, al invertir inicio y objetivo cicla entre dos celdas durante 500 iteraciones. El Bug2 completo evita el ciclo con la m-line, pero sigue siendo subóptimo (1307 vs 338 en la casa).

3. A* y el distance transform coinciden en longitud, no en camino. Ambos dan 32 pasos por rutas distintas (arriba vs borde superior-izquierdo). A* busca solo lo necesario; el DT precomputa la solución para todas las celdas.

Conclusiones (2/2)

4. La heurística euclidiana es admisible pero poco informativa en 4-conectividad. A* expande el 85 % de las celdas libres. Una heurística Manhattan seguiría siendo admisible y expandiría menos, sin perder optimalidad.

5. El límite fijo de 100 barridos no escala. En 20 × 20 sobran 95 barridos; en la casa del libro (397 × 596) se necesitan 571. Hay que detectar la convergencia o adaptar el número de barridos al tamaño del mapa.

6. La parte E (ROS2 / Nav2) queda fuera de esta entrega. A–D cubren navegación reactiva, evasión, planificación y mapas de ocupación. La integración en el stack de Nav2 se deja para una entrega posterior.

Idea de cierre: no hay un algoritmo "mejor". Hay un compromiso entre información disponible, cómputo y garantías — y cada parte del laboratorio mueve una de esas tres piezas.

Trabajo futuro

Corto plazo

- Parte E: llevar A* y el DT a **Nav2** con un robot simulado
- Repetir A con la ley del libro y comparar la convergencia
- Heurística Manhattan en A*: medir cuántas expansiones ahorra

Mediano plazo

- Detección de convergencia en el DT en lugar de 100 barridos fijos
- Extender la métrica a **8 vecinos** (Chebyshev / euclidiana)
- Bug2 completo con m-line y comparación cuantitativa

4	12	1
actividades implementadas	preguntas respondidas	comando para reproducir todo

Referencias

Corke, P. (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python* (3rd ed.). Springer. Capítulo 5: §5.1.1 (Braitenberg), §5.1.2 (Bug2), §5.3 (A*), §5.4.1 (distance transform).

Guía de laboratorio de simulación, Capítulo 5 — Corke. Universidad Nacional de Colombia, sede Manizales, 2026.

Corke, P. *RVC3-python*: notebook `notebooks/chap5.ipynb` y modelo `RVC3/models/braitenberg.py`.

Resultados locales: `Laboratorio/resultados/metricas.csv` y `Laboratorio/figuras/`.

Reproducir todo (desde `Laboratorio/`, con el entorno del curso):

```
source ~/venvs/robotica/bin/activate
python experimentos/run_all.py
```

¿Preguntas?

Gracias

Juan David · Robótica · UNAL Manizales · 2026-2